

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P1380R1
Date:	2019-01-20
Audience	CWG, LWG
Revises:	P1380R0
Author:	Lawrence Crowl
Contact	Lawrence@Crowl.org

Ambiguity and Insecurities with Three-Way Comparison

Lawrence Crowl, Lawrence@Crowl.org

Abstract

Three-way comparisons are currently unsound. There are problems in the definition of three-way comparison functions and in the definition of template non-type parameter equivalence. We propose several small changes to improve the soundness of comparison.

Contents

[Introduction](#)

[General Comparison Functions](#)

[16.11.4 Comparison algorithms \[cmp.alg\]](#)

[23.7.11 Three-way comparison algorithms \[alg.3way\]](#)

[Floating-Point Comparison Functions](#)

[16.11.4 Comparison algorithms \[cmp.alg\]](#)

[Non-Type Equivalence and Structural Equality](#)

[12.5 Type equivalence \[temp.type\]](#)

[10.10.1 Defaulted comparison operator functions \[class.compare.default\]](#)

[Revisions](#)

Introduction

[P0100R0 Comparison in C++](#) introduced three-way comparison with the express purpose of making comparison sound. Programmers should be sure that their comparison operations meet the constraints of the algorithms they use.

Recent changes to the standard have made comparison less sound. Much of the problem seems to have arisen with the 'demotion' of the three-way comparison functions from specialization points providing the primary comparison mechanism to adjuncts to the three-way comparison operators.

Existing algorithms currently take a boolean comparison function. For compatibility with existing programs, these algorithms will continue to accept boolean comparisons. When we introduce three-way comparison into algorithms, we can do so in a sound manner without invalidating any existing code.

All wording edits are relative to [N4778 Working Draft, Standard for Programming Language C++](#).

General Comparison Functions

Comparison operators can serve two purposes, the application domain or the C++ standard algorithms. When these two purposes cannot be simultaneously satisfied, as with floating-point computation, the operators should be left to the application domain. To quote from [P0100R2 Comparison in C++](#), "robust standard algorithms should never use [boolean] comparison operators". The consequence is that (a) C++ standard algorithms must accept three-way comparison functions and (b) those functions should not be 'automatically' defined in terms of boolean comparison operators. However, the current definitions do define those functions in terms of boolean comparison operators. The intent of this paper is to change those definitions to reduce or expose errors.

The current definitions of several standard functions infer a strong ordering from existing `<` and `==` operators. Unfortunately, doing so has a strong chance of making the program unsound. For example, see Herb Sutter's `CaseInsensitiveString` ([P0515R0 Consistent comparison](#)). If that class were written with traditional operators, the existing `strong_order` function would silently promote the weak ordering to strong. A strong ordering on user-defined types takes special care to ensure substitutability. As a consequence, it is easy to write a class that fails to provide strong ordering. So, we should not infer a strong ordering from non-3way operators.

In contrast, user-defined types are much more likely to provide a weak ordering on existing operators, if for no other reason than to enable use with the standard algorithms. So, inferring a weak ordering from existing operators is far less problematic. However, there is still the potential to strengthen a partial ordering into a weak ordering. As currently defined, this strengthening causes silent failure. An exception is preferable to silent failure.

16.11.4 Comparison algorithms [cmp.alg]

Constructing a strong ordering from the existence of `<` and `==` has a strong chance of making the program unsound. Furthermore, most algorithms do not need a strong ordering, so the risk outweighs the reward. Programmers that need the strong ordering can write it.

Remove paragraph 1.4.

~~(1.4) — Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then~~

- ~~• (1.4.1) — if `a == b` is true, returns `strong_ordering::equal`;~~
- ~~• (1.4.2) — otherwise, if `a < b` is true, returns `strong_ordering::less`;~~
- ~~• (1.4.3) — otherwise, returns `strong_ordering::greater`.~~

Rather than silently promote partial orders to weak orders, we propose to detect comparisons that are not weak and throw an exception.

Edit paragraph 2.3 as follows.

(2.3) — Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then

- (2.3.1) — if `a == b` is true, returns `weak_ordering::equivalent`;
- (2.3.2) — otherwise, if `a < b` is true, returns `weak_ordering::less`;
- (2.3.3) — otherwise, if `b < a` is true, returns `weak_ordering::greater`;
- (2.3.4) — otherwise, throws `std::domain_error`.

Similarly, the definition for `partial_order` incorrectly strengthens a partial ordering.

Edit paragraph 3.3 as follows.

(3.3) — Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, then

- (3.3.1) — if `a == b` is true, returns `partial_ordering::equivalent`;
- (3.3.2) — otherwise, if `a < b` is true, returns `partial_ordering::less`;
- (3.3.3) — otherwise, if `b < a` is true, returns `partial_ordering::greater`;
- (3.3.4) — otherwise, returns `partial_ordering::unordered`.

Similarly to `strong_order`, `strong_equal` unsafely strengthens the order of the operators.

Delete paragraph (4.3).

~~(4.3) — Otherwise, if the expression `a == b` is well-formed and convertible to `bool`, then~~

- ~~• (4.3.1) — if `a == b` is true, returns `strong_equality::equal`;~~
- ~~• (4.3.2) — otherwise, returns `strong_equality::nonequal`.~~

Paragraph 5 defining `weak_order` is probably an acceptable default. Programmers that use operator `==` for other purposes will need to delete the overload.

23.7.11 Three-way comparison algorithms [alg.3way]

As before, inferring a `strong_ordering` or a `strong_equality` from `<` and `==` is unsound. Again, a weak ordering, with protection, is probably acceptable.

Edit paragraph 1.2 for `compare_3way` as follows.

(1.2) — Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, returns `strong_ordering::equal` `weak_ordering::equivalent` when `a == b` is true,
otherwise returns `strong_ordering::less` `weak_ordering::less` when `a < b` is true, and
otherwise returns `strong_ordering::greater` `weak_ordering::greater` when `b < a` is true,
and otherwise throws `std::domain_error`.

(1.3) — Otherwise, if the expression `a == b` is well-formed and convertible to `bool`, returns `strong_equality::equal` `weak_equality::equivalent` when `a == b` is true, and otherwise
returns `strong_equality::nonequal` `weak_equality::nonequivalent`.

[P1186R0 When do you actually use `<=>`](#) proposes to move this function definition into the definition for operator `<=>`. If so done, the edits here would apply to operator `<=>`.

Floating-Point Comparison Functions

The boolean comparison operators for standard floating-point types have a partial order, so we have no reason to assume that non-standard floating-point types have stronger orders. Despite this, non-standard floating-point types are not explicitly handled.

There are reasons to wish for a weak ordering on floating-point types, particularly when keeping sets of 'normally distinguishable' values. We provide the means to do so.

16.11.4 Comparison algorithms [cmp.alg]

The `strong_order` function has a special case for ISO/IEC/IEEE floating point. However, when `is_iec559` is false, there is no special handling of floating point. The default algorithm may lead to intransitive 'greater' relations. We propose to explicitly provide for non-standard floating point.

Edit paragraph 1.1 as follows.

(1.1) — If `is_floating_point<T>::value` is true, returns a result of type `strong_ordering` that is a strong order and that is consistent with T's comparison operators. If `numeric_limits<T>::is_iec559` is also true, ~~returns a result of type `strong_ordering` that~~ order must be consistent with the `totalOrder` operation as specified in ISO/IEC/IEEE 60559.

Because standard floating point provides only a partial ordering, we have a gap in the comparison strengths at weak ordering. So, a similar special case is needed for a weak ordering. This special case should be consistent ([P0100R2 Comparison in C++, Consistency Between Relations](#)) with both `strong_order` and the partial order implied by the operators.

Add a new subparagraph before (2.1) as follows.

(2.05) — If `is_floating_point<T>::value` is true, returns a result of type `weak_ordering` that is a weak order and is consistent with T's comparison operators and with `strong_order`. If `numeric_limits<T>::is_iec559` is also true, returns a result of type `weak_ordering` that has the following equivalence classes ordered from lesser to greater.

- Together, all negative NaN values.
- Negative infinity.
- Separately, each normal and subnormal negative value.
- Together, both zero values.
- Separately, each normal and subnormal positive value.
- Positive infinity.
- Together, all positive NaN values.

Non-Type Equivalence and Structural Equality

The definition of non-type template parameter equivalence appeals to the `<=>` operator, but fails to specify the strength. Specifying a strong ordering solves the problem.

Note that using `strong_compare` or `strong_equal` instead of operator `<=>` for non-type template argument equivalence would enable more types as non-type template parameters, particularly floating-point types. We make no proposal here, but merely float the idea. Doing so requires stable conversion of decimal literals to floating-point values.

12.5 Type equivalence [temp.type]

Template non-type argument equality is ambiguous; it fails to specify the strength of the equality. Since this ambiguity affects the application binary interface, we should be cautious and require strong equality.

Edit paragraph 1.5 as follows.

(1.5) — their remaining corresponding non-type *template-arguments* have the same type and value after conversion to the type of the *template-parameter*, where they are considered to have the same value if they compare ~~equal~~ `std::strong_ordering::equal` with operator<=>, and

[P1185R0](#) `<=> != ==` proposes a "structural equality operator" (section 4.1 of the paper, section 10.10.1 of the standard). With adoption of [P1185R0](#) or a successor, the above paragraph should be rewritten using strong structural equality.

Edit paragraph 1.5 as follows.

(1.5) — their remaining corresponding non-type *template-arguments* have the same type and value after conversion to the type of the *template-parameter*, where they are considered to have the same value if they compare equal with [a strong structural equality](#) operator<=>, and

10.10.1 Defaulted comparison operator functions [class.compare.default]

[P1185R0](#) `<=> != ==` proposes a "structural equality operator" (section 4.1 of the paper, section 10.10.1 of the standard). This proposal calls out floating-point types as an exception, rather than calling out the fact that the comparison is not strong. This approach means that the standard has a stealth defect if it ever introduces a new built-in type without strong equality.

Edit the proposal in [P1185R0](#) section 4.1 as follows.

An == (equal to) operator is a structural equality operator if:

- it is a built-in candidate ([over.built]) where ~~neither argument has floating point type~~ [both arguments have strong equality comparisons](#), or
- it is an operator for a class type C that is defined as defaulted in the definition of C and all == operators it invokes are structural equality operators.

A type T has strong structural equality if, for a glvalue x of type const T, x == x is a valid expression of type bool and invokes a structural equality operator.

Revisions

From [P1380R0](#):

- Add contents.
- Simplify the abstract.

- Reorganize the presentation to topics, rather than sections in the standard.
- Add introductory background material.
- Define orderings for non-IEEE floating point types.
- Improve exposition.
- Correct bugs.
- Remove the aside on the confusion of "explicitly defaulted functions defined as deleted".
- Add a revision section.